Cuestionario de Evaluación · Implantación de Arquitecturas Web

1. Los tres elementos fundamentales del esquema de funcionamiento de los servicios web

El esquema de funcionamiento de los servicios web requiere tres elementos. El proveedor del servicio web lo diseña, desarrolla e implementa y lo pone disponible para su uso, dentro de la organización o en público. El consumidor del servicio es quien accede al componente para utilizar sus servicios. El agente del servicio es el enlace entre proveedor y consumidor, encargado de publicar, buscar y localizar el servicio.

El servicio no se presta ni se consume directamente entre dos interlocutores: hay un tercero, el agente, que lo publica, lo busca y lo localiza para que el consumidor lo encuentre.

2. Las tres capas de un modelo de arquitectura web

La arquitectura web es un modelo compuesto de tres capas.

Atendiendo al despliegue y a la tecnología, esas capas son la capa de base de datos, que reúne toda la información que se administra mediante el servicio web (almacén, estructura y recuperación de los datos) y se apoya en gestores como MySQL o PostgreSQL; la capa de servidores de aplicaciones web, que ejecuta las aplicaciones y gestiona las solicitudes y respuestas HTTP, con ejemplos como Apache, Tomcat o Resin; y la capa de clientes, punto de acceso del usuario al servicio, materializada en los navegadores.

Atendiendo a la responsabilidad funcional, las capas son la de presentación, que se ocupa de la navegabilidad, la validación de los datos de entrada, el formateo de los datos de salida y la presentación de la web, y es la que se presenta al usuario; la de negocio, que recibe las peticiones del usuario, envía las respuestas y verifica que las reglas establecidas se cumplen; y la de acceso a datos, formada por los gestores que almacenan, estructuran y recuperan los datos solicitados por la capa de negocio.

La misma arquitectura se nombra también navegador web, tecnología web dinámica (PHP, Java Servlets, ASP) y base de datos.

La separación en capas aporta escalabilidad, separación de responsabilidades, portabilidad y utilización de componentes, y permite una separación física de las capas (una máquina por capa mediante middlewares), que es la base de la escalabilidad vertical.

3. Diferencia entre PHP/ASP y JavaScript: ciclo de vida y lugar de ejecución

La diferencia es dónde se ejecuta el código: en el servidor (PHP y ASP) o en el cliente (JavaScript).

Aspecto	PHP / ASP (servidor)	JavaScript (cliente)
Lugar de ejecución	En el servidor web o de aplicaciones.	En el navegador del equipo cliente.
Cuándo se ejecuta	Tras cada petición: el servidor invoca el intérprete antes de enviar nada.	Después de descargarse la página: el motor de JavaScript del navegador la analiza y ejecuta.
Qué viaja por la red	El resultado ya generado (HTML final).	El código fuente del script, visible para el cliente.
Acceso a datos	Puede consultar y modificar la base de datos y el sistema de archivos del servidor.	Solo accede a lo que se le ha enviado; para pedir más datos hace una nueva petición (AJAX o fetch).
Ciclo de vida	Por petición: el script se ejecuta, produce la respuesta y termina; el estado se conserva con sesión.	Por página: desde la carga del documento hasta su cierre, manteniendo estado entre eventos.
Visibilidad y seguridad	El código no llega al cliente, así que oculta datos y reglas de negocio.	El código es visible en el código fuente, así que no sirve para ocultar información sensible.

ASP (Active Server Pages) ejecuta las páginas activas en el servidor y genera a partir de ellas los resultados que se muestran en el navegador de cada cliente. El código que llega con la página y debe ejecutarse, normalmente JavaScript, lo ejecuta el propio navegador. Ambos son lenguajes interpretados, pero la diferencia no está en el lenguaje sino en el punto de ejecución: PHP y ASP generan la respuesta en el servidor; JavaScript solo da comportamiento a la página en el cliente.

4. Diferencias entre el Modelo 1 y el Modelo 2

La evolución va separando responsabilidades y pasa por cuatro modelos: Modelo 1, Modelo 1.5, Modelo 2 y Modelo 2X.

Aspecto	Modelo 1	Modelo 2
Base tecnológica	Modelo web CGI: procesos externos al servidor web.	Patrón MVC, con un controlador de la navegación.
Dónde vive cada responsabilidad	Presentación, negocio y acceso a datos se confunden en un mismo script (Perl o PHP).	Separadas lógicamente: el controlador dirige, las páginas JSP hacen de presentación y los JavaBeans encapsulan el negocio. Los beans siguen incrustados en las JSP, así que la separación es de responsabilidades, no de ficheros.
Generación de la salida	El script produce el HTML que recibe el navegador.	Las vistas JSP se procesan en el servidor y generan el HTML.
Unidad de código	Cada página es un programa completo que mezcla las tres capas.	El controlador es único y centraliza las peticiones; el código se reparte en clases y páginas.
Mantenimiento	Difícil: un cambio afecta a toda la página y hay que repetir lógica.	Sencillo: cada pieza tiene una responsabilidad clara y se reutiliza y prueba por separado.

El Modelo 1.5 es el escalón intermedio propio de Java: aparecen las JSP y los servlets, la presentación recae en las JSP y los beans incrustados se encargan del negocio y del acceso a datos, pero todavía no hay un controlador que centralice la navegación. El Modelo 2 añade ese controlador, que es el Servlet Controlador. El Modelo 2X se orienta a aplicaciones multicanal (una PDA, un terminal móvil o cualquier navegador HTML) y usa plantillas XSL para transformar datos XML y publicar la misma aplicación en distintos dispositivos. La diferencia estructural clave es la separación de responsabilidades: en el Modelo 1 las tres capas conviven en un mismo script; en el Modelo 2 cada una tiene su componente.

5. Los componentes de las plataformas LAMP y WISA

Una plataforma web se define por cuatro componentes: sistema operativo, servidor web, gestor de bases de datos y lenguaje de programación interpretado.

Componente	LAMP (software libre)	WISA (software propietario)
Sistema operativo	Linux	Windows
Servidor web	Apache	Internet Information Services (IIS)
Gestor de bases de datos	MySQL	SQL Server
Lenguaje de backend	PHP (a veces Perl o Python)	ASP o ASP.NET

La diferencia de fondo es la licencia: LAMP usa componentes de software libre, sin las restricciones propietarias de WISA, que se basa en tecnologías de Microsoft. Ambas plataformas cierran con el mismo esquema de tres niveles: navegador web, tecnología web dinámica (PHP, Java Servlets, ASP) y base de datos encargada de almacenar de forma permanente y actualizada la información.

6. Balanceador de carga hardware tradicional frente a balanceador hardware HTTP

La escalabilidad horizontal consiste en añadir otra máquina de características similares y balancear la carga con un dispositivo externo, que puede ser de tres tipos:

Tipo	Modo de operación	Inspecciona el paquete HTTP	Mantiene la relación usuario-máquina	Velocidad
Balanceador software	Servidor web como Apache con mod_jk (o mod_proxy y mod_proxy_balancer) que redirige las peticiones entre las máquinas.	Sí, e identifica la sesión registrando qué máquina la atiende.	Sí, por afinidad de sesión (sticky sessions).	La más lenta.
Balanceador hardware tradicional	Dispositivo que responde solo a algoritmos de reparto (Round Robin, LRU) y conmuta circuitos; reenvía la petición sin mirar su contenido.	No: no examina ni interpreta el paquete HTTP.	No: no garantiza que la misma sesión vaya a la misma máquina.	El más rápido.
Balanceador hardware HTTP	Dispositivo hardware que sí examina el paquete HTTP para decidir el nodo y mantiene la relación usuario-máquina.	Sí.	Sí.	Muy rápido: más que el software y algo menos que el hardware puro.

El hardware tradicional y el hardware HTTP se diferencian en el modo de operación, la inspección y su efecto sobre el diseño. En el modo de operación, el tradicional conmuta circuitos y aplica el reparto sin interpretar la conversación, mientras que el hardware HTTP interpreta el protocolo de aplicación y puede descartar, redirigir o reescribir antes de enviar. En la inspección, el tradicional no examina el paquete HTTP y por tanto ignora quién es el usuario, mientras que el hardware HTTP lo identifica y mantiene la relación usuario-máquina. Como consecuencia, el tradicional no garantiza que la misma sesión caiga en la misma máquina, lo que obliga a guardar la sesión en cookies o en base de datos; el hardware HTTP evita esa penalización. En cuanto al coste, el hardware HTTP es mucho más rápido que los balanceadores software, aunque algo menos que el hardware puro.

Visto por niveles OSI, el tradicional opera en capa 4 (conmutación de conexiones) y el hardware HTTP en capa 7 (inspección del protocolo de aplicación); el balanceador software, que también inspecciona el HTTP, queda en capa 7 con el coste de CPU de una máquina de propósito general.

7. Las tres categorías de módulos del servidor web Apache

El servidor está estructurado en módulos: cada módulo agrupa funciones de un aspecto concreto, y el binario httpd trae módulos compilados que se activan o desactivan al arrancar. Hay tres categorías. Los módulos base cubren las funciones básicas (core, siempre disponible en la distribución). Los

módulos multiproceso se encargan de la unión de los puertos de la máquina, es decir, de aceptar y atender las peticiones: son los MPM prefork, worker y event, con directivas comunes en mpm_common. Los módulos adicionales añaden funcionalidad: mod_alias, mod_rewrite, mod_ssl, mod_status, mod_proxy, mod_proxy_ajp, mod_proxy_balancer y mod_session.

El índice oficial de Apache 2.4 agrupa de forma algo distinta: separa Core Features and Multi-Processing Modules (core, mpm_common y los MPM) de Other Modules (donde están los adicionales). Other Modules no es una categoría funcional y algunos módulos básicos, como mod_authz_core, mod_mime o mod_unixd, aparecen en ese segundo bloque. Además, mod_so carga módulos compartidos (DSO) al arrancar o al reiniciar, y mod_unixd aporta la seguridad básica en Unix y el cambio de privilegios de los procesos hijo.

Se habilitan y deshabilitan con a2enmod y a2dismod (Debian y Ubuntu) o con directivas LoadModule en apache2.conf:

```
sudo a2enmod proxy proxy_ajp proxy_balancer
```

8. Privilegios de root en el puerto 80 y atención posterior a los clientes

Los puertos por debajo del 1024 están reservados a los procesos del sistema y solo root puede hacer bind sobre ellos. Si la directiva Listen usa el 80, o cualquier puerto por debajo del 1024, hace falta root para iniciar Apache.

El proceso principal (httpd) arranca como root, abre sus ficheros de log y lanza varios procesos hijo, que escuchan y atienden las peticiones. El principal sigue como root, pero los hijos se ejecutan con menos privilegios (www-data en Debian y Ubuntu, apache en Red Hat), según las directivas User y Group. El reparto es seguro porque el proceso que no atiende clientes es el único con root y los que atienden peticiones arbitrarias no tienen privilegios: una vulnerabilidad en un hijo no compromete la máquina y, si un hijo muere, el principal crea otro. Los hijos comparten el mismo usuario sin privilegios, así que el aislamiento entre ellos es limitado; la protección real es que ninguno es root.

El servicio se gestiona con apachectl (start, restart, stop), que traduce esos argumentos a señales para httpd. Apache solo aplica los cambios de configuración al reiniciar.

9. Los tres modos de despliegue de un contenedor de servlets

Un contenedor de servlets maneja e invoca servlets por cuenta del usuario. Hay tres modos según dónde se ejecute la máquina virtual Java respecto al servidor web.

Modo	Cómo funciona	Rendimiento	Escalabilidad y estabilidad
1. Stand-alone (independiente)	El contenedor forma parte integral del servidor web; es el caso de un servidor web basado en Java (JavaWebServer, sustituido por iPlanet). Por defecto Tomcat trabaja así.	—	—
2. Dentro de proceso (in-process)	Un plugin para el servidor web más una implementación de contenedor Java. El plugin abre una JVM dentro del espacio de direcciones del servidor web y pasa al contenedor, usando JNI, las peticiones que ejecutan servlets.	Alto: adecuado para servidores multihilo de un solo proceso.	Limitado: mejor rendimiento, escalabilidad reducida.
3. Fuera de proceso (out-of-process)	Un plugin para el servidor web más una implementación de contenedor Java que se ejecuta en una JVM fuera del servidor web; se comunican por IPC, normalmente sockets TCP/IP.	Menor: el tiempo de respuesta no es tan bueno como el anterior.	Mejor: gana sobre todo en escalabilidad y estabilidad.

Tomcat puede usarse como contenedor solitario (desarrollo y depuración) o como plugin de un servidor web existente (Apache o IIS). Con los modos 2 o 3 hay que instalar además un adaptador de servidor web.

10. Protocolo AJP: puerto habitual y ventajas frente a HTTP simple

El protocolo es AJP (Apache JServ Protocol), implementado en Apache por mod_proxy_ajp, un módulo de soporte de AJP para mod_proxy. El puerto habitual es el 8009, puerto de trabajo por defecto del conector AJP de Tomcat (configurable en conf/server.xml); en Apache se declara con BalancerMember ajp://localhost:8009.

AJP es un protocolo interno y rápido que usa conexiones TCP persistentes y comunica Apache con Tomcat.

Frente a HTTP simple tiene tres ventajas. Las conexiones TCP persistentes: mantiene conexiones abiertas en lugar de abrir y cerrar una por petición, ahorrando el handshake TCP. Las cabeceras pre-empacadas en formato binario: los nombres más comunes son enteros y el resto cadenas con longitud, de modo que no se analizan ni se reconstruyen en cada salto y se reduce la CPU. La menor latencia resultante, sobre todo con muchas peticiones pequeñas.

Conviene precisar dos puntos. El cuerpo de la petición sí se copia: viaja en un paquete aparte, en bloques de 8 KB (packetSize 8192), que el contenedor pide con GET_BODY_CHUNK; lo que AJP evita es el escaping de caracteres en URLs y cookies, no la copia del cuerpo. Y AJP no multiplexa: una conexión se asigna a una petición hasta que termina, por eso mantiene varias conexiones abiertas, una por petición en vuelo.

No es un protocolo estándar para clientes: es interno entre el servidor web y el contenedor de servlets. El puerto 8009 no debe exponerse a Internet; Tomcat moderno trae secretRequired en true, que impide arrancar el conector si no se define un secret compartido. El secreto viaja en el campo ? secret del paquete AJP13_FORWARD_REQUEST y se declara en Apache con secret=CLAVE (en ProxyPass o BalancerMember; exige httpd 2.4.42 o superior).

La puesta en marcha carga proxy, proxy_ajp y proxy_balancer, define el BalancerMember con protocolo, IP y puerto, y usa ProxyPreserveHost On para mantener la cabecera Host original. La configuración mínima es:

```
<VirtualHost *:80>
    ServerName www.ejemplo.com
    ProxyPreserveHost On
    ProxyPass        / http://localhost:8009/
    ProxyPassReverse / http://localhost:8009/
</VirtualHost>
```

11. Flujo de control de un Servlet Controlador en una arquitectura MVC

El Servlet Controlador es el controlador de navegación del Modelo 2 y todas las peticiones de la aplicación pasan por él:

1. El navegador envía una petición HTTP (GET o POST) a una URL de la aplicación.
2. El contenedor (Tomcat) la resuelve contra el descriptor WEB-INF/web.xml, que mapea el patrón de URL con el servlet declarado (servlet, servlet-name, servlet-class y servlet-mapping).
3. El contenedor crea una instancia del servlet (o reutiliza una del pool) y llama a service(), que delega en doGet() o doPost() según el método HTTP.
4. El controlador extrae los parámetros con request.getParameter() y, si hace falta, del objeto de sesión.
5. Delega la lógica en el modelo (JavaBeans o clases de negocio); el controlador no contiene reglas de negocio, solo coordina.
6. Coloca el resultado en el ámbito de la petición o de la sesión, para que la vista lo lea.
7. Elige la vista y devuelve su nombre lógico, no su ruta física.
8. Reenvía (forward) con RequestDispatcher a la JSP; el reenvío es interno y la URL del navegador no cambia.
9. Tomcat compila la JSP a servlet con Jasper y la ejecuta, componiendo el HTML con los atributos (EL o JSTL).
10. El servidor devuelve el HTML al navegador.

Los filters pueden actuar antes y después del controlador (autenticación, logging, control de acceso) sin mezclarse con el MVC. En resumen, el controlador recibe la petición, extrae los datos, llama al modelo, coloca el resultado, elige la vista y reenvía a ella; la vista solo pinta y el modelo solo aplica reglas y accede a los datos. Es el patrón Front Controller; una implementación conocida es el DispatcherServlet de Spring.

12. Round Robin frente a LRU en servidores web balanceados

Round Robin reparte de forma cíclica y sin memoria (A, B, C, A...). Es stateless, determinista y barato, pero ignora la carga real: solo reparte peticiones, no trabajo.

LRU (Least Recently Used) recuerda en qué máquina se atendió a cada usuario y elige la del último uso más antiguo; no reparte por carga sino por recencia. Aporta afinidad de sesión sin interpretar la petición, pero es más complejo y caro porque mantiene una tabla de estado en cada petición. Un nodo con una petición larga y reciente consta como usado aunque esté saturado; para repartir por carga existen otros métodos, como least connections o least response time.

Con peticiones concurrentes, el estado que LRU necesita choca con la naturaleza del balanceador. Necesita identificar al cliente y el balanceador tradicional no puede, porque solo ve IP y puerto y no garantiza la afinidad; dos peticiones simultáneas de la misma sesión pueden acabar en nodos distintos y romper la HttpSession; la tabla usuario-máquina es un punto único de fallo que además diverge entre instancias del balanceador; y un nodo ocupado por una petición larga no se libra de trabajo. Los fallos típicos de estas tablas son el reparto desigual tras proxys, la afinidad con direcciones IP dinámicas y la pérdida de afinidad si la tabla se desborda.

Apache resuelve lo mismo con los métodos de reparto de mod_proxy_balancer: byrequests (cuenta de peticiones, equivalente a Round Robin), bybusyness (el nodo con menos peticiones pendientes, que sí tiene en cuenta la carga), bytraffic y heartbeat. Los tres últimos mantienen estado, y por eso el balanceador software es más lento que un dispositivo dedicado.

En resumen, Round Robin es simple y escalable pero reparte a ciegas; LRU da afinidad de sesión a costa de identificar al usuario y mantener estado, algo inviable en un balanceador que no inspecciona el HTTP. En la práctica, o se usa un balanceador que sí inspecciona el HTTP, o la sesión se guarda fuera (cookies, base de datos o almacén compartido).

13. JNI: papel en la integración de componentes y su impacto en la estabilidad

JNI (Java Native Interface) es la interfaz estándar de la JVM que permite a código nativo (C o C++) invocar y manipular código Java, y viceversa. Su papel es el de puente entre el mundo Java y el nativo dentro del mismo espacio de direcciones. El espacio de direcciones compartido es propio del modo dentro de proceso (el plugin carga la JVM como una biblioteca del proceso del servidor web), no de JNI, que tampoco es la única técnica de interoperabilidad. Se usa en ese modo dentro de proceso: el plugin abre una JVM dentro del proceso del servidor web y le pasa, usando JNI, las peticiones que ejecutan servlets. El mecanismo se apoya en un puntero de interfaz (JNIEnv) con el que se obtienen objetos, se invocan métodos, se crean excepciones y se liberan referencias.

El impacto en la estabilidad es muy negativo. No hay aislamiento: el código nativo comparte proceso y espacio de direcciones con el servidor, y un fallo grave o una caída (segfault) se lleva por delante todo el proceso. No hay excepciones que lo protejan: los errores de memoria en C son segmentation faults o corrupción, no excepciones Java, y la especificación advierte de que pasar punteros ilegales o argumentos incorrectos puede corromper el sistema o la VM. Las fugas de referencias y el uso desde varios hilos producen fugas de memoria o comportamiento no determinista, ya que el puntero de interfaz solo es válido en su hilo. Tampoco hay red de seguridad ante la concurrencia: si dos hilos usan un mismo recurso nativo, el resultado es corrupción de datos o un fallo del proceso, y por eso la

especificación exige compilar las librerías nativas con soporte multihilo (-mt, -D_REENTRANT). Por último, la escalabilidad es menor.

En resumen, JNI da acceso al código nativo sin capas de traducción y permite reutilizar librerías nativas, pero no acelera por sí solo y sacrifica la estabilidad: un solo fallo del componente nativo puede tumbar el servicio. Por eso el modo fuera de proceso cambia tiempo de respuesta por más escalabilidad y estabilidad: si la JVM nativa cae, Apache sobrevive.

14. Replicación de sesión en un cluster de servidores de aplicaciones

Una sesión HTTP vive en memoria del servidor que la creó (HttpSession en Java) y, en un cluster, cada petición puede caer en una máquina distinta que no la tenga. La replicación de sesión mantiene una copia del estado de la sesión compartida entre todos los nodos, de modo que cualquier máquina puede atender la petición. Por la propia replicación, estos sistemas suelen presentar problemas de rendimiento.

En Tomcat, el elemento Cluster activa la replicación de sesiones, la de atributos del contexto y el despliegue de WAR en todo el cluster. Por debajo hay un Session Manager (por defecto DeltaManager, que envía solo los cambios y va acoplado a SimpleTcpCluster), un Channel llamado Tribes (comunicación y pertenencia al cluster) y un Valve que decide cuándo replicar: siempre al final de la petición, no en cada modificación. El atributo channelSendOptions decide el envío: asynchronous (8, por defecto; más rendimiento pero posible desorden en el receptor), use_ack (2, confirma recepción) y synchronized_ack (4, confirma recepción y procesamiento; requiere use_ack), a cambio de más tráfico y latencia.

Es un cuello de botella por varias razones. Por el coste O(n) por escritura: cada cambio se serializa y se envía a los n−1 nodos, de modo que añadir máquinas empeora el rendimiento. Por la duplicación de memoria: cada nodo guarda las sesiones de todos los usuarios. Por la latencia y la contención que añaden la serialización y la transmisión en la ruta crítica. Por el coste de serializar el estado, sobre todo si hay objetos no serializables. Y por el riesgo de perder cambios no confirmados durante la resincronización.

Como alternativas: un almacén externo compartido (Redis o Memcached) ofrece una fuente de verdad única sin tráfico O(n); la afinidad de sesión (sticky sessions) reduce el tráfico pero pierde equilibrio de carga y complica el failover; el balanceador y mod_session permiten definir el almacén de sesiones en Apache (cookie, base de datos, DBD); y un diseño sin sesión (stateless, por ejemplo con JWT) deja que cualquier nodo atienda cualquier petición.

La replicación se activa declarando el Cluster en conf/server.xml, y el secret del conector AJP debe coincidir con el de Apache:

```
<Connector port="8009" protocol="AJP/1.3" secretRequired="true" secret="CLAVE"/>

<Cluster className="org.apache.catalina.ha.tcp.SimpleTcpCluster"
         channelSendOptions="6">
    <Manager className="org.apache.catalina.ha.session.DeltaManager"/>
</Cluster>
```